

Universidad La Salle
Facultad de Ingenierías y Arquitectura
Departamento Académico de Ingeniería y Matemáticas
Carrera Profesional de Ingeniería de Software

Ingeniería Web

SGCP

Sistema de Gestión de Citas para Peluquería

NUEVO Video del informe:

<https://youtu.be/wzdrU1Fdn8>

Autor	Rol
Davis Arapa Chua	Configuración del proyecto Django y estructura base
Sebastian Castro Mamani	Restricciones del modelo y pruebas funcionales
Piero Delgado Chipana	Administración de modelos en Django Admin
Hugo Diaz Chavez	Documentación y evidencias

29 de mayo de 2026

Índice

1. Descripción del Proyecto	2
2. Equipos, Materiales y Tecnologías Utilizadas	2
3. URL del Repositorio GitHub	2
4. Estructura del Proyecto	2
5. Preparación del Entorno para Django y JWT	4
5.1. Marco Teórico	4
5.2. Configuración del Entorno Virtual	4
5.3. Instalación de Dependencias	4
5.4. Generación de Migraciones y Superusuario	4
6. Modelo de Datos: Diagrama E-R	5
6.1. Entidades del Sistema	5
7. Inicialización del Servidor	6
7.1. Credenciales del Superusuario	6
7.2. Creación de un Superusuario	6
8. Pruebas de la API REST con JWT	7
8.1. Autenticación con JWT	7
8.2. Validación de Seguridad	8
8.3. Operaciones CRUD sobre Peluquerías	9
9. Frontend: Interfaz Web Estática	11
9.1. Arquitectura del Frontend	12
9.2. Habilitación de CORS	12
9.3. Uso del Frontend	12
10.Base de Datos PostgreSQL	12
11.Conclusiones	13
12.Rúbrica	13
13.Entregables	14

1. Descripción del Proyecto

El presente sistema web, denominado **SGCP** (Sistema de Gestión de Citas para Peluquería), tiene el objetivo de automatizar la generación y gestión de citas en establecimientos de peluquería. El sistema permite:

- Registrar peluquerías, usuarios (clientes, recepcionistas y administradores), estilistas y servicios.
- Gestionar la disponibilidad horaria de cada estilista por día de la semana.
- Reservar citas verificando la ausencia de cruces de horario.
- Exponer una API REST completa protegida con autenticación JWT.
- Administrar todos los recursos desde el panel de Django Admin.

2. Equipos, Materiales y Tecnologías Utilizadas

Herramienta	Versión	Uso
Windows / Linux	—	Sistema operativo de desarrollo
Visual Studio Code	1.9x	Editor principal y cliente REST
Python	3.11+	Lenguaje principal
Django	6.0.4	Framework web backend
Django REST Framework	3.17.1	Construcción de la API RESTful
SimpleJWT	5.5.1	Autenticación con tokens JWT
django-cors-headers	4.7+	Habilitación de CORS para el frontend
HTML / CSS / JS	—	Frontend estático (consumo directo de la API)
SQLite	—	Base de datos local (desarrollo)
PostgreSQL	12+	Base de datos de producción
Git	—	Control de versiones
GitHub	—	Repositorio remoto colaborativo
django-extensions	4.1	Generación del diagrama E-R

3. URL del Repositorio GitHub

Repositorio GitHub:

<https://github.com/Vsrn12/SGCP.git>

4. Estructura del Proyecto

```
1 SGCP /
2 |--- README.md
3 |--- requirements.txt
4 |
5 |--- Back/
6 |   |--- db/
7 |   |   |--- SGCPelu.sql           # Backup completo PostgreSQL
```

```

8 | | |--- peluqueria_db.sql      # Script de tablas
9 | | |--- datos_prueba.sql     # Datos de prueba
10 | |
11 | |--- MyDjangoProject/
12 | |   |--- manage.py
13 | |   |--- db.sqlite3
14 | |   |--- crear_datos.py
15 | |   |--- MyDjangoProject/
16 | |   |--- settings.py        # +corsheaders, +rest_framework
    | |   , +simplejwt
17 | |   |--- urls.py            # Router API + /api/token/
18 | |   |--- wsgi.py
19 | |   |--- asgi.py
20 | |
21 | |   |--- MyWebApps/
22 | |       |--- SAC/
23 | |           |--- admin.py
24 | |           |--- serializers.py
25 | |           |--- views.py
26 | |           |--- migrations/
27 | |           |--- models/
28 | |               |--- base.py
29 | |               |--- peluqueria.py
30 | |               |--- usuario.py
31 | |               |--- estilista.py
32 | |               |--- servicio.py
33 | |               |--- cita.py
34 | |               |--- horario_estilista.py
35 |
36 |--- Front/
37 |   |--- index.html          # Pantalla de login
38 |   |--- dashboard.html     # Dashboard principal (6 tabs CRUD)
39 |   |--- app.http            # Pruebas REST (VS Code REST Client)
40 |   |--- css/
41 |   |   |--- style.css      # Estilos: login, navbar, tablas,
    |   |   modales
42 |   |   |--- js/
43 |   |       |--- auth.js    # Login/logout, refresco de token JWT
44 |   |       |--- api.js     # fetch wrapper con Bearer token
    |   |       automatico
45 |   |       |--- app.js     # Logica del dashboard
46 |
47 |--- Informes/
48 |   |--- informe.tex        # Este documento

```

Listing 1: Estructura de directorios

5. Preparación del Entorno para Django y JWT

5.1. Marco Teórico

Django es un framework web de alto nivel basado en Python que promueve el desarrollo rápido, limpio y pragmático. Utiliza el patrón Modelo-Plantilla-Vista (MTV).

Para este proyecto se complementa con:

- **Django REST Framework:** Extiende Django para construir APIs RESTful robustas con serialización, validación y autenticación integradas.
- **SimpleJWT:** Implementa autenticación basada en tokens JWT (JSON Web Tokens), permitiendo autorización sin estado y mejorando la seguridad de la API.
- **SQLite:** Sistema de base de datos compacto ideal para desarrollo, fácilmente reemplazable por PostgreSQL en producción.

5.2. Configuración del Entorno Virtual

```
1 $ cd SGCP
2 $ python -m venv venv
3 $ source venv/bin/activate           # Linux/macOS
4 $ venv\Scripts\Activate.ps1         # Windows PowerShell
```

Listing 2: Configuración del entorno virtual

5.3. Instalación de Dependencias

```
1 (venv) $ pip install Django==6.0.4
2 (venv) $ pip install djangorestframework
3 (venv) $ pip install djangorestframework-simplejwt
4 (venv) $ pip install django-cors-headers
5 (venv) $ pip install django-extensions
6 (venv) $ pip install pydotplus
7 (venv) $ pip install sqlparse==0.5.5
```

Listing 3: Instalación de dependencias

```
1 (venv) $ pip freeze > requirements.txt
```

Listing 4: Generación de requirements.txt

5.4. Generación de Migraciones y Superusuario

```
1 (venv) $ cd Back/MyDjangoProyect
2 (venv) $ python manage.py makemigrations
3 (venv) $ python manage.py migrate
```

Listing 5: Generando y aplicando migraciones

```

1 % El superusuario ya esta incluido en db.sqlite3 del repositorio.
2 % Credenciales de acceso:
3 %   Usuario:      admin
4 %   Contraseña: 123
5 % Si se necesita recrear:
6 (venv) $ python manage.py createsuperuser

```

Listing 6: Creación del superusuario

Campo	Valor
Usuario	admin
Contraseña	123

Cuadro 1: Credenciales del superusuario (entorno de desarrollo)

```

1 (venv) $ python manage.py graph_models SAC -o sac.dot
2 (venv) $ dot -Tpng sac.dot -o sac.png

```

Listing 7: Generando diagrama E-R

6. Modelo de Datos: Diagrama E-R

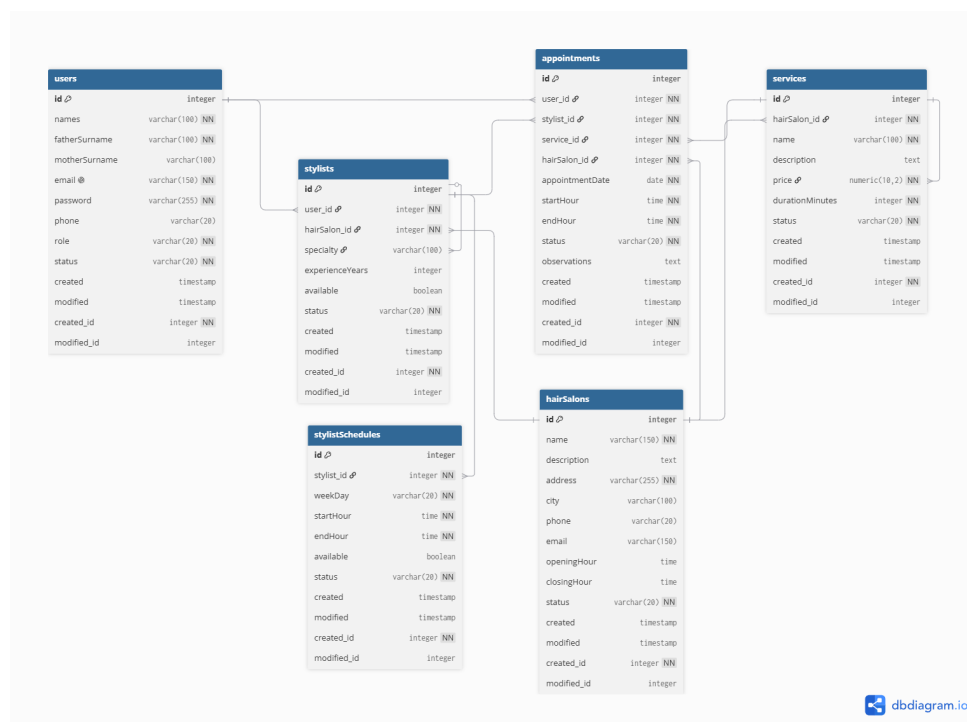


Figura 1: Mi imagen

6.1. Entidades del Sistema

Peluqueria (hair_salons): Representa la sede principal del negocio. Contiene nombre, dirección, teléfono y descripción.

Usuario (users): Almacena a los clientes, recepcionistas y administradores. La contraseña se almacena cifrada mediante `make_password`. Los roles válidos son: `cliente`, `administrador`, `recepcionista`.

Estilista (stylists): Modela al personal que atiende los servicios. Incluye nombre, teléfono, especialidad y la peluquería a la que pertenece.

Servicio (services): Define la oferta comercial. Contiene nombre, descripción, precio y duración en minutos.

Cita (appointments): Gestiona las reservas. Asocia usuario, estilista y servicio en una fecha y horario específicos. El estado puede ser: `pendiente`, `confirmada`, `completada`, `cancelada`.

HorarioEstilista (stylist_schedules): Describe la disponibilidad semanal de cada estilista con día, hora de inicio y hora de fin.

Todos los modelos heredan de **ModeloAuditable**, que centraliza los campos de auditoría: `status`, `created`, `modified`, `created_id`, `modified_id`.

7. Inicialización del Servidor

```
1 $ git clone https://github.com/Vsrn12/SGCP.git
2 $ cd SGCP
3 $ cd Back/MyDjangoProyect
4 $ python -m venv venv
5 $ source venv/bin/activate          # Linux/macOS
6 $ venv\Scripts\Activate.ps1        # Windows
7 (venv) $ pip install -r ../../requirements.txt
8 (venv) $ python manage.py migrate
9 (venv) $ python manage.py runserver
```

Listing 8: Clonando e iniciando el servidor

7.1. Credenciales del Superusuario

La base de datos `db.sqlite3` incluida en el repositorio ya contiene el superusuario de desarrollo:

Campo	Valor
Usuario	admin
Contraseña	123

Cuadro 2: Credenciales del superusuario (entorno de desarrollo)

7.2. Creación de un Superusuario

```
1 (venv) $ python manage.py createsuperuser
2 Nombre de usuario: admin
3 Password: ****
4 Superuser created successfully.
```

Listing 9: Creando un superusuario

8. Pruebas de la API REST con JWT

Para validar el correcto funcionamiento del sistema, se empleó el cliente **VS Code REST Client** (Front/app.http) para realizar pruebas sobre los endpoints.

8.1. Autenticación con JWT

Se envía una petición POST /api/token/ con credenciales del superusuario:

```
1 POST http://127.0.0.1:8000/api/token/
2 Content-Type: application/json
3
4 {
5     "username": "admin",
6     "password": "admin1234"
7 }
```

Listing 10: Obtener token JWT — Solicitud

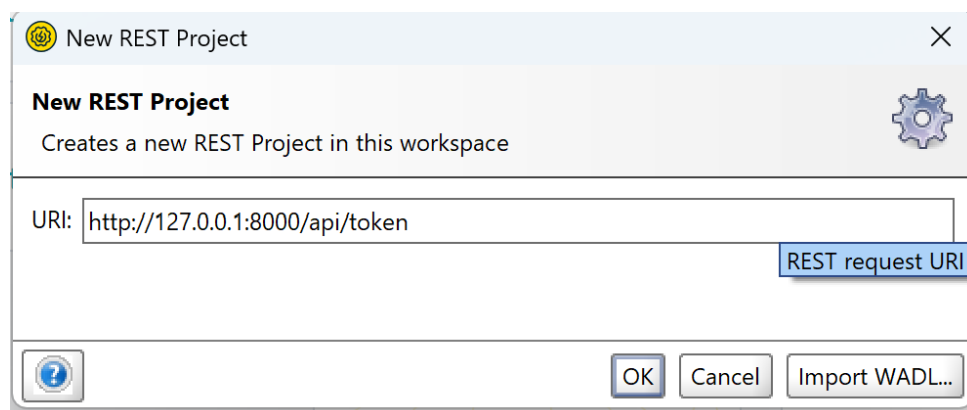


Figura 2: Login JWT (api/token)

```
1 {
2     "refresh": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... ",
3     "access": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... "
4 }
```

Listing 11: Obtener token JWT — Respuesta

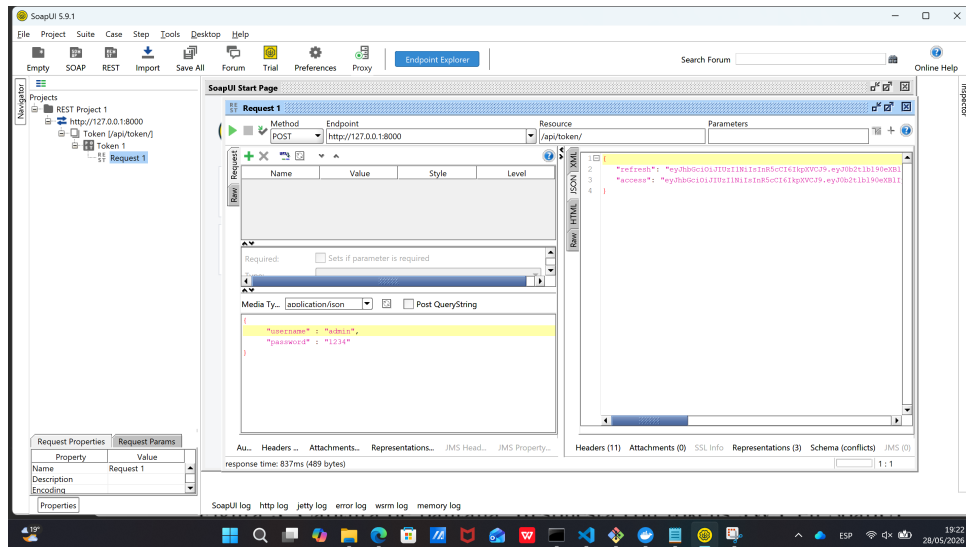


Figura 3: Respuesta con tokens JWT en SoapUI

8.2. Validación de Seguridad

Se verificó que la API rechaza peticiones sin autenticación:

```
1 GET http://127.0.0.1:8000/api/peluquerias/
```

Listing 12: Acceso sin token — Solicitud

```
1 {
2   "detail": "Authentication credentials were not provided."
3 }
```

Listing 13: Acceso sin token — Respuesta (HTTP 401)

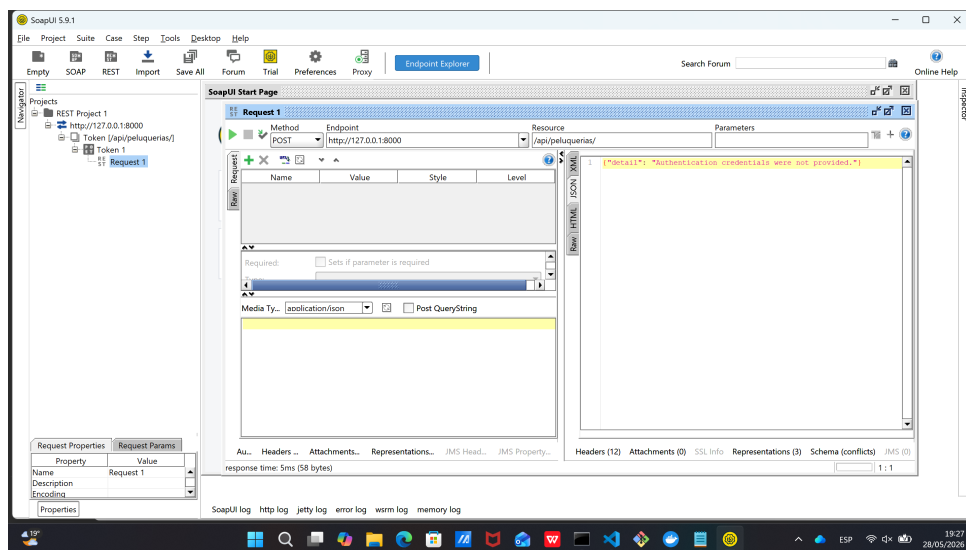


Figura 4: Solicitud de listado de peluquerias en SoapUI (Sin token)

8.3. Operaciones CRUD sobre Peluquerías

Listar peluquerías — Con el header `Authorization: Bearer <token>`, la petición `GET /api/peluquerias/` devuelve la lista en JSON (HTTP 200).

```
1 GET http://127.0.0.1:8000/api/peluquerias/  
2 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Listing 14: Listar peluquerías — Solicitud

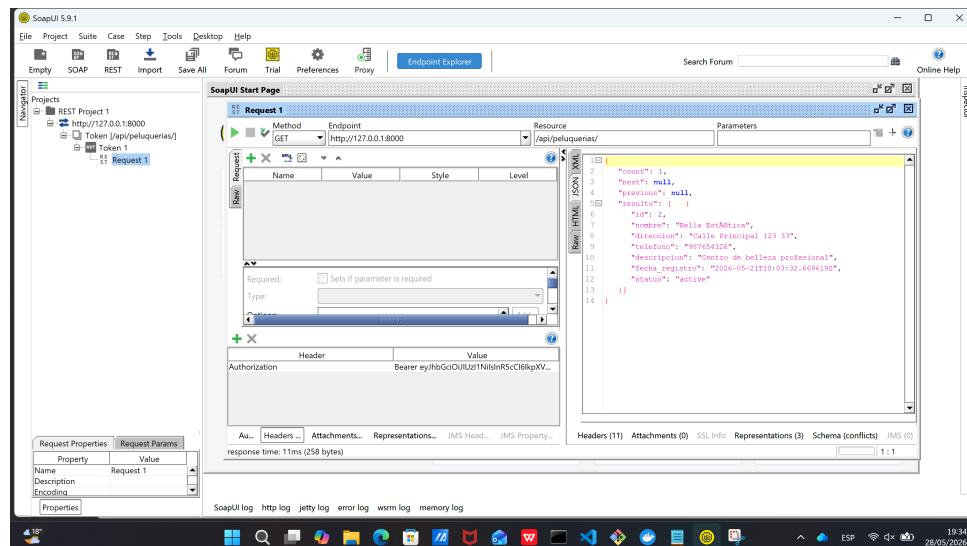


Figura 5: Listado de peluquerias en SoapUI

Crear peluquería — `POST /api/peluquerias/` con cuerpo JSON genera un nuevo registro (HTTP 201).

```
1 POST http://127.0.0.1:8000/api/peluquerias/  
2 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...  
3 Content-Type: application/json  
4  
5 {  
6     "nombre": "Bella Estetica",  
7     "direccion": "Av. Principal 123",  
8     "telefono": "987654321",  
9     "descripcion": "Centro de belleza profesional",  
10    "status": "active"  
11 }
```

Listing 15: Crear peluquería — Solicitud

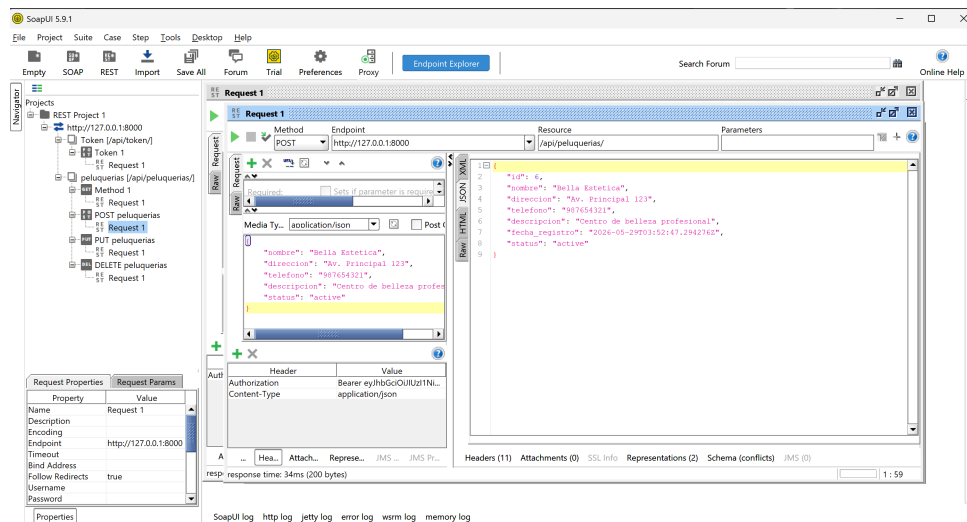


Figura 6: Creación de peluquería en SoapUI

Actualizar peluquería — PUT /api/peluqueras/id/ permite modificar la información de una peluquería existente mediante el envío de un cuerpo JSON con los nuevos datos. La operación devuelve una respuesta HTTP 200 indicando que la actualización fue realizada correctamente.

```

1 PUT http://127.0.0.1:8000/api/peluqueras/1/
2 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
3 Content-Type: application/json
4
5 {
6   "nombre": "Bella Estetica Premium",
7   "direccion": "Av. Principal 456",
8   "telefono": "999888777",
9   "descripcion": "Centro de belleza y spa profesional",
10  "status": "active"
11 }

```

Listing 16: Actualizar peluquería — Solicitud

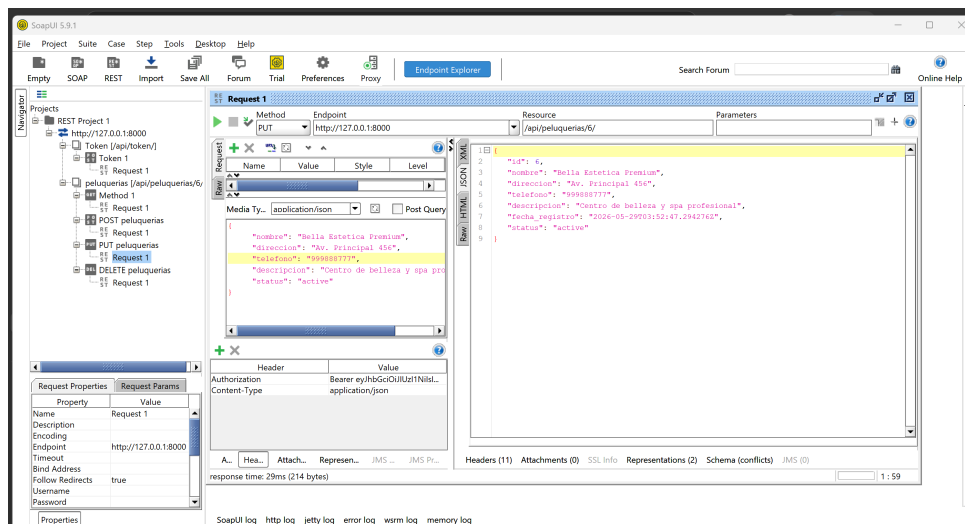


Figura 7: Actualización de peluquería en SoapUI

Eliminar peluquería — DELETE `/api/pelequerias/id/` elimina un registro existente de la base de datos utilizando el identificador de la peluquería. Si la operación se realiza correctamente, el servidor devuelve una respuesta HTTP 204 indicando que el recurso fue eliminado satisfactoriamente.

```
1 DELETE http://127.0.0.1:8000/api/pelequerias/6/
2 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Listing 17: Eliminar peluquería — Solicitud

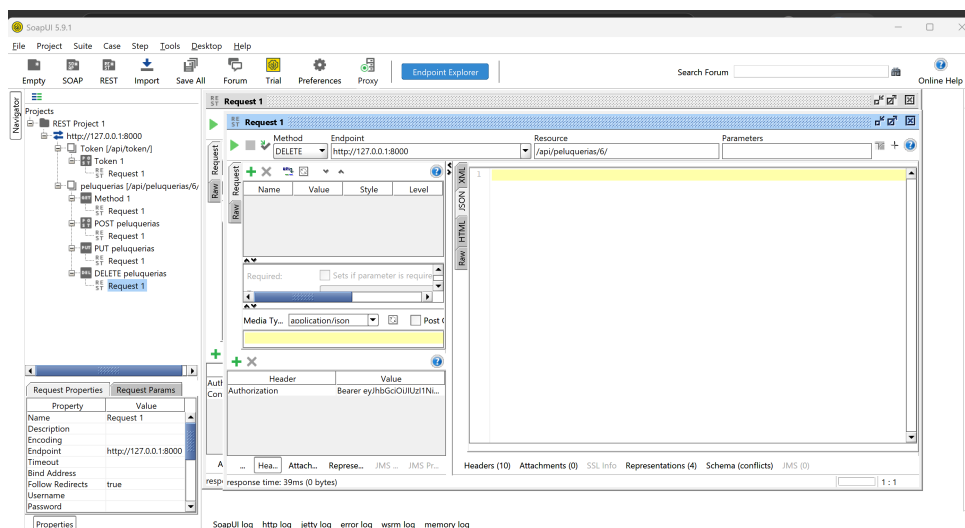


Figura 8: Eliminación de peluquería en SoapUI

9. Frontend: Interfaz Web Estática

El directorio `Front/` contiene una interfaz web estática desarrollada en HTML, CSS y JavaScript puro que consume la API REST directamente desde el navegador.

9.1. Arquitectura del Frontend

- **auth.js:** Gestiona el ciclo de vida del token JWT. Realiza el POST `/api/token/`, almacena `access` y `refresh` en `localStorage`, y refresca el token automáticamente cuando expira (POST `/api/token/refresh/`).
- **api.js:** Wrapper sobre `fetch` que adjunta el header `Authorization: Bearer <token>` en cada petición y reintenta con el nuevo token si recibe HTTP 401.
- **app.js:** Lógica del dashboard. Gestiona la navegación por tabs, renderiza tablas dinámicas y abre modales de creación y edición para las seis entidades del sistema.

9.2. Habilitación de CORS

Para que el navegador pueda realizar peticiones al servidor Django desde un origen distinto (archivo local o servidor estático), se configuró `django-cors-headers` en `settings.py`:

```
1 INSTALLED_APPS = [  
2     ...  
3     'corsheaders',  
4     ...  
5 ]  
6  
7 MIDDLEWARE = [  
8     'corsheaders.middleware.CorsMiddleware', # debe ir primero  
9     'django.middleware.security.SecurityMiddleware',  
10    ...  
11 ]  
12  
13 # Desarrollo: permite cualquier origen  
14 CORS_ALLOW_ALL_ORIGINS = True
```

Listing 18: Configuración CORS en settings.py

9.3. Uso del Frontend

1. Iniciar el servidor Django (`python manage.py runserver`).
2. Abrir `Front/index.html` en el navegador (Live Server o doble clic).
3. Iniciar sesión con las credenciales del superusuario.
4. Navegar por los tabs: Peluquerías, Estilistas, Servicios, Usuarios, Citas, Horarios.

10. Base de Datos PostgreSQL

El directorio `Back/db/` contiene los archivos SQL para el entorno de producción:

- **SGCPelu.sql:** Backup completo con estructura y datos de prueba.
- **peluqueria_db.sql:** Script de creación de tablas.

- **datos_prueba.sql**: Datos de prueba adicionales.

```

1 psql -U postgres -h localhost -c "CREATE DATABASE sgcpelu;"
2 psql -U postgres -h localhost -d sgcpelu -f Back/db/SGCPelu.sql
3 psql -U postgres -h localhost -d sgcpelu -c "\dt"

```

Listing 19: Importar backup PostgreSQL

11. Conclusiones

- La autenticación mediante JWT se implementó correctamente, permitiendo un acceso seguro y controlado a los recursos de la API REST.
- Todas las operaciones CRUD fueron implementadas, probadas con el cliente REST de VS Code (**app.http**) y accesibles desde el frontend web.
- Se desarrolló una interfaz web estática (HTML/CSS/JS) que consume la API REST con autenticación JWT, sin necesidad de frameworks adicionales de frontend.
- El sistema rechaza correctamente las solicitudes no autenticadas, validando la seguridad de los endpoints.
- Los modelos incluyen validaciones de negocio robustas: control de cruces de horario, encriptación de contraseñas y consistencia de relaciones entre entidades.
- La arquitectura modular con modelos separados y herencia de **ModeloAuditable** facilita el mantenimiento y extensión del sistema.

12. Rúbrica

Ítem	Puntos	Descripción
1. GitHub	2	Repositorio con todos los archivos. Se clona.
2. Base de datos	4	Django Admin + SQLite/PostgreSQL configurados
3. REST Framework	4	Endpoints serializados JSON. CRUD completo
4. JWT	4	Autenticación con SimpleJWT
5. REST Client	3	Pruebas con VS Code REST Client (app.http)
6. Deploy	3	Despliegue en la nube (pendiente)
Total	20	

13. Entregables

- Informe en formato L^AT_EX (`Informes/informe.tex`).
- Repositorio GitHub clonable con README.md y todos los archivos del proyecto:
<https://github.com/Vsrn12/SGCP.git>
- Proyecto listo para despliegue (pendiente configurar en la nube).
- Video de demostración y prueba de funcionalidad: *[pendiente de enlace]*.